

Singular Value Decomposition and PCA from Scratch

Vatsav Reddy

Under the Guidance of
Prof. Praveen Parchuri

International Institute of Information Technology, Hyderabad

May 30, 2026

Contents

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

Variance: Measuring Spread of a Single Variable

What is Variance and Why Do We Need It?

Before we can understand PCA, we need to understand **variance**. Imagine measuring the heights of 100 students. Some are tall, some are short — the *variance* tells us numerically how “spread out” the heights are around the average.

Formally, for a random variable x with mean $\mu_x = \mathbb{E}[x]$:

$$\text{Var}(x) = \mathbb{E}[(x - \mu_x)^2]$$

We square the deviation $(x - \mu_x)$ so that positive and negative deviations do not cancel each other.

Expanding the Formula (Useful Shortcut)

$$\mathbb{E}[(x - \mu_x)^2] = \mathbb{E}[x^2 - 2\mu_x x + \mu_x^2] = \mathbb{E}[x^2] - 2\mu_x \underbrace{\mathbb{E}[x]}_{\mu_x} + \mu_x^2 = \mathbb{E}[x^2] - \mu_x^2$$

This gives the useful identity: $\text{Var}(x) = \mathbb{E}[x^2] - (\mathbb{E}[x])^2$.

Sample Variance (From Finite Data)

Given n observations $x_1, \dots, x_n$ with sample mean $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

The factor $n - 1$ (Bessel's correction) makes s^2 an *unbiased* estimator. In machine learning we often use $\frac{1}{n}$ for computational convenience.

Covariance: How Two Variables Move Together

Motivation

Variance handles *one* variable. In real datasets we have many features and we want to know: does a high value of feature x tend to come with a high or low value of feature y ? This is captured by **covariance**.

Population Covariance

For two random variables x and y with means μ_x, μ_y :

$$\text{Cov}(x, y) = \mathbb{E}[(x - \mu_x)(y - \mu_y)] = \mathbb{E}[xy] - \mathbb{E}[x]\mathbb{E}[y]$$

Why does this measure joint movement? When $x > \mu_x$ and $y > \mu_y$ simultaneously (both above average), the product $(x - \mu_x)(y - \mu_y) > 0$. If this happens often, the expectation is positive. Conversely, if one goes up while the other goes down, the product is negative.

Sample Covariance and Interpretation

$$\widehat{\text{Cov}}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

- **Positive covariance:** x and y tend to increase *together*
- **Negative covariance:** one increases as the other decreases
- **Near zero:** no detectable linear relationship
- **Special case:** $\text{Cov}(x, x) = \text{Var}(x)$ — covariance of a variable with itself is its variance

Correlation: Scale-Free Covariance

Why Covariance Alone Is Not Enough

Covariance depends on the *units* of x and y . Height measured in metres vs. centimetres will give different covariance values even if the underlying relationship is the same. We need a unit-free measure.

Pearson Correlation Coefficient

Normalise covariance by the product of standard deviations:

$$\rho_{xy} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}, \quad \sigma_x = \sqrt{\text{Var}(x)}, \quad \sigma_y = \sqrt{\text{Var}(y)}$$

$$-1 \leq \rho_{xy} \leq 1$$

This range is guaranteed by the Cauchy–Schwarz inequality.

Sample version:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Interpretation and Connection to PCA

$\rho = +1$: perfect positive linear relationship. $\rho = 0$: no linear relationship. $\rho = -1$: perfect negative linear relationship.

High $|\rho|$ means one variable is approximately a linear function of the other — so one feature is *redundant*. **PCA's goal is to find a new set of features where the correlation (off-diagonal covariance) between every pair is exactly zero.**

The Covariance Matrix: Capturing All Pairwise Relationships

Setup: Many Variables at Once

Suppose each data point is a vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^n$. Instead of computing $\binom{n}{2}$ individual covariances, we collect them all into one object: the **covariance matrix**.

$$\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top], \quad \boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$$

Entry (i, j) : $\Sigma_{ij} = \text{Cov}(x_i, x_j)$. Diagonal entries are variances; off-diagonal are covariances.

Explicit 2×2 Example

For $\mathbf{x} = [x, y]^\top$:

$$\Sigma = \begin{bmatrix} \text{Var}(x) & \text{Cov}(x, y) \\ \text{Cov}(y, x) & \text{Var}(y) \end{bmatrix}$$

The matrix is **symmetric** because $\text{Cov}(x_i, x_j) = \text{Cov}(x_j, x_i)$, i.e. $\Sigma = \Sigma^\top$.

A *diagonal* covariance matrix means all features are uncorrelated — PCA's goal is to find a transformation that makes this happen.

Correlation Matrix

Divide each entry by $\sigma_i \sigma_j$: $R_{ij} = \frac{\text{Cov}(x_i, x_j)}{\sigma_i \sigma_j}$, so $R_{ii} = 1$. The diagonal is all ones, off-diagonal entries are Pearson correlations.

Sample Covariance Matrix from Data

From Data Matrix to Covariance Matrix

We have m data points in $\mathbb{R}^n$, arranged as rows of $X \in \mathbb{R}^{m \times n}$ (each row = one data point, each column = one feature).

Step 1 — Centre the data: $X_c = X - \mathbf{1}_m \boldsymbol{\mu}^\top$ where $\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i$. After centring, each column of X_c has zero mean.

Step 2 — Compute the sample covariance matrix:

$$\Sigma = \frac{1}{m} X_c^\top X_c \in \mathbb{R}^{n \times n}$$

Why does this formula work? Entry (j, k) of $X_c^\top X_c$ is $\sum_{i=1}^m (X_c)_{ij} (X_c)_{ik}$, and dividing by m gives exactly the sample covariance between feature j and feature k .

Two Key Properties (Proved Directly)

Symmetric: $\Sigma^\top = \frac{1}{m} (X_c^\top X_c)^\top = \frac{1}{m} X_c^\top X_c = \Sigma$ (using $(AB)^\top = B^\top A^\top$).

Positive semi-definite (PSD): For any $\mathbf{v} \in \mathbb{R}^n$:

$$\mathbf{v}^\top \Sigma \mathbf{v} = \frac{1}{m} \mathbf{v}^\top X_c^\top X_c \mathbf{v} = \frac{1}{m} \|X_c \mathbf{v}\|^2 \geq 0$$

A squared norm is always non-negative, so all eigenvalues $\lambda_i \geq 0$. Variance cannot be negative — this is the matrix generalisation of that fact.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

Eigenvectors and Eigenvalues: The Core Idea

Definition: Vectors That Only Scale, Never Rotate

When a matrix multiplies a general vector, the vector typically changes *both* its direction and its magnitude — it gets “rotated and stretched”.

An **eigenvector** $\mathbf{v}$ of a square matrix $A \in \mathbb{R}^{n \times n}$ is a special vector that *only gets scaled* (its direction stays the same or flips):

$$A\mathbf{v} = \lambda \mathbf{v}$$

λ is the corresponding **eigenvalue**. Positive λ means scaling; negative means flipping direction; $|\lambda| > 1$ means stretching; $|\lambda| < 1$ means compression.

How to Find Eigenvalues

Rearrange: $(A - \lambda I)\mathbf{v} = \mathbf{0}$. For a nonzero solution $\mathbf{v}$ to exist, the matrix $(A - \lambda I)$ must be singular:

$$\det(A - \lambda I) = 0$$

This is the **characteristic polynomial** in λ . Its roots are the eigenvalues. For each eigenvalue, solve $(A - \lambda I)\mathbf{v} = \mathbf{0}$ to find the eigenvector.

Key Insight: Matrix Multiplication Becomes Scalar Multiplication

If $\{\mathbf{v}_i\}$ are eigenvectors forming a basis, and $\mathbf{x} = \sum_{i=1}^n \alpha_i \mathbf{v}_i$, then:

$$A\mathbf{x} = A \sum_i \alpha_i \mathbf{v}_i = \sum_i \alpha_i A\mathbf{v}_i = \sum_i \alpha_i \lambda_i \mathbf{v}_i$$

The expensive matrix multiplication reduces to n cheap scalar multiplications $\alpha_i \mapsto \alpha_i \lambda_i$. This is the fundamental computational power of eigenvectors.

Eigenvalue Decomposition (EVD): Matrix Form

Stacking Eigenvectors into a Matrix

Let A have n linearly independent eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ with eigenvalues $\lambda_1, \dots, \lambda_n$. Form the matrix $V = [\mathbf{v}_1 \cdots \mathbf{v}_n] \in \mathbb{R}^{n \times n}$ and the diagonal matrix $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$.

$$AV = A[\mathbf{v}_1 \cdots \mathbf{v}_n] = [\lambda_1 \mathbf{v}_1 \cdots \lambda_n \mathbf{v}_n] = [\mathbf{v}_1 \cdots \mathbf{v}_n] \text{diag}(\lambda_1, \dots, \lambda_n) = V\Lambda$$

Since V is invertible (eigenvectors are linearly independent):

$$A = V\Lambda V^{-1}$$

This is the **Eigenvalue Decomposition** of A . Multiplying by V^{-1} on the right and V on the left gives the diagonalisation $\Lambda = V^{-1}AV$.

The Spectral Theorem: Why Symmetric Matrices Are Special

For a **real symmetric** matrix $A = A^T$:

- ➊ All eigenvalues are **real** (no complex numbers)
- ➋ Eigenvectors for distinct eigenvalues are **orthogonal**: $\mathbf{v}_i^T \mathbf{v}_j = 0$ for $i \neq j$
- ➌ We can choose eigenvectors to have unit length, so $V^T V = I$, meaning $V^{-1} = V^T$

Therefore, for real symmetric A :

$$A = V\Lambda V^T \quad (\text{Spectral Decomposition})$$

The inverse is free: $V^{-1} = V^T$.

Applying EVD to the Covariance Matrix Σ

Diagonalising Σ : What We Get

Since $\Sigma = \frac{1}{m} X_c^\top X_c$ is real, symmetric, and PSD, the Spectral Theorem guarantees:

$$\Sigma \mathbf{p}_i = \lambda_i \mathbf{p}_i, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0, \quad \mathbf{p}_i^\top \mathbf{p}_j = \delta_{ij}$$

In matrix form:

$$\Sigma = P \Lambda P^\top, \quad P = [\mathbf{p}_1 \ \dots \ \mathbf{p}_n], \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$$

Why Eigenvalues Equal the Variance Along Each Direction

The variance of the projected data along direction $\mathbf{p}_i$ is:

$$\frac{1}{m} (X_c \mathbf{p}_i)^\top (X_c \mathbf{p}_i) = \frac{1}{m} \mathbf{p}_i^\top X_c^\top X_c \mathbf{p}_i = \mathbf{p}_i^\top \Sigma \mathbf{p}_i = \mathbf{p}_i^\top (\lambda_i \mathbf{p}_i) = \lambda_i \underbrace{\|\mathbf{p}_i\|^2}_{=1} = \lambda_i$$

The eigenvalue λ_i is exactly the variance of the projected data. Large λ_i means direction $\mathbf{p}_i$ captures substantial spread in the data.

Practical Computation

Use `numpy.linalg.eigh` (not `eig`) for symmetric matrices. It returns *real* eigenvalues in ascending order and is more numerically stable. Reverse the output to get descending order (largest variance first).

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost**
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

The Computational Problem with Direct EVD

Why We Cannot Directly Diagonalise Σ for Images

Consider 100×100 pixel face images: each image is a vector of $n = 10,000$ numbers (one per pixel).

The covariance matrix $\Sigma = \frac{1}{m} X_c^\top X_c$ lives in $\mathbb{R}^{n \times n} = \mathbb{R}^{10,000 \times 10,000}$.

- **Memory:** Storing Σ requires $n^2 = 10^8$ floating-point numbers ≈ 800 MB
- **Eigendecomposition cost:** $O(n^3) = O(10^{12})$ operations — completely impractical

Yet we have, say, only $m = 400$ training images. We need a smarter approach.

The Gram Matrix: A Much Smaller Object

Instead of diagonalising the $n \times n$ matrix Σ , we diagonalise the $m \times m$ **Gram matrix**:

$$G = \frac{1}{m} X_c X_c^\top \in \mathbb{R}^{m \times m}$$

With $m = 400$ training images, the EVD of G costs $O(m^3) = O(400^3) = O(6.4 \times 10^7)$ operations.

Speedup: $\left(\frac{n}{m}\right)^3 = \left(\frac{10,000}{400}\right)^3 = 25^3 = 15,625 \times$ faster.

Why G and Σ Share the Same Non-Zero Eigenvalues

Key Theorem: Shared Spectrum

The non-zero eigenvalues of $G = \frac{1}{m} X_c X_c^\top$ and $\Sigma = \frac{1}{m} X_c^\top X_c$ are **identical**. This is a standard result about matrices of the form AB and BA .

Here we prove it constructively — and the proof also tells us *how to recover the eigenvectors of Σ from those of G* .

Proof: From an Eigenvector of G to an Eigenvector of Σ

Suppose $G\mathbf{v}_i = \lambda_i \mathbf{v}_i$ with $\lambda_i \neq 0$ and $\|\mathbf{v}_i\| = 1$. Expand:

$$\frac{1}{m} X_c X_c^\top \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

Multiply both sides on the **left** by $X_c^\top$:

$$X_c^\top \left(\frac{1}{m} X_c X_c^\top \mathbf{v}_i \right) = X_c^\top (\lambda_i \mathbf{v}_i)$$

$$\frac{1}{m} \underbrace{X_c^\top X_c}_{\text{candidate}} \underbrace{X_c^\top \mathbf{v}_i}_{\text{candidate}} = \lambda_i \underbrace{X_c^\top \mathbf{v}_i}_{\text{candidate}}$$

Recognising $\Sigma = \frac{1}{m} X_c^\top X_c$:

$$\boxed{\underbrace{\Sigma (X_c^\top \mathbf{v}_i)}_{\text{candidate}} = \lambda_i \underbrace{(X_c^\top \mathbf{v}_i)}_{\text{candidate}}}$$

So $X_c^\top \mathbf{v}_i$ is an eigenvector of Σ with eigenvalue λ_i . □

Normalising the Pixel-Space Eigenvector: Full Derivation

Why We Must Normalise

We have shown that $\mathbf{p}_i \propto X_c^\top \mathbf{v}_i$. But PCA requires unit-norm principal directions ($\|\mathbf{p}_i\| = 1$). We must divide by the norm. Let us compute it exactly.

Computing $\|X_c^\top \mathbf{v}_i\|$ Step by Step

$$\begin{aligned}\|X_c^\top \mathbf{v}_i\|^2 &= (X_c^\top \mathbf{v}_i)^\top (X_c^\top \mathbf{v}_i) && \text{(definition of squared norm)} \\ &= \mathbf{v}_i^\top X_c X_c^\top \mathbf{v}_i && \text{(transpose distributes: } (AB)^\top = B^\top A^\top \text{)} \\ &= \mathbf{v}_i^\top (mG) \mathbf{v}_i && \text{(since } X_c X_c^\top = mG \text{ by definition)} \\ &= m \mathbf{v}_i^\top G \mathbf{v}_i && \text{(pull out scalar } m \text{)} \\ &= m \mathbf{v}_i^\top (\lambda_i \mathbf{v}_i) && \text{(since } G \mathbf{v}_i = \lambda_i \mathbf{v}_i \text{)} \\ &= m \lambda_i \underbrace{\mathbf{v}_i^\top \mathbf{v}_i}_{=1} && (\mathbf{v}_i \text{ is unit norm)} \\ &= m \lambda_i\end{aligned}$$

Therefore $\|X_c^\top \mathbf{v}_i\| = \sqrt{m \lambda_i}$.

Final Formula for the Normalised Eigenface

$$\mathbf{p}_i = \frac{X_c^\top \mathbf{v}_i}{\sqrt{m \lambda_i}}, \quad \|\mathbf{p}_i\| = 1$$

Interpretation: $\mathbf{v}_i \in \mathbb{R}^m$ gives the coefficients for linearly combining the m training images. $X_c^\top \mathbf{v}_i$ converts that combination into a pixel pattern. After dividing by $\sqrt{m \lambda_i}$, we get a unit-norm direction in pixel space — this is an **eigenface**.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline**
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

PCA: Step-by-Step Algorithm with Reasoning

Step 1 — Centre the Data (Remove the Mean)

Compute the mean image: $\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i \in \mathbb{R}^n$.

Centre: $X_c = X - \mathbf{1}_m \boldsymbol{\mu}^\top$. Now every column of X_c has zero mean.

Why is centring necessary? Covariance measures spread *around the mean*. If we do not subtract the mean, $\Sigma = \frac{1}{m} X^\top X$ measures spread around the origin — a meaningless quantity unless the data already has zero mean. Centring ensures Σ correctly captures how features vary relative to their averages.

Step 2 — Compute Eigenvectors via the Gram Matrix Trick

- 1 Compute $G = \frac{1}{m} X_c X_c^\top \in \mathbb{R}^{m \times m}$ (cheap: only $m \times m$)
- 2 Solve $G \mathbf{v}_i = \lambda_i \mathbf{v}_i$ (cost: $O(m^3)$ instead of $O(n^3)$)
- 3 Recover pixel-space eigenvectors via $\mathbf{p}_i = \frac{X_c^\top \mathbf{v}_i}{\sqrt{m \lambda_i}}$
- 4 Order by $\lambda_1 \geq \lambda_2 \geq \dots$ (descending variance)

Step 3 — Select Top- k Principal Components

Form the matrix of the top k eigenvectors as *rows*:

$$P_k = \begin{bmatrix} \mathbf{p}_1^\top \\ \mathbf{p}_2^\top \\ \vdots \\ \mathbf{p}_k^\top \end{bmatrix} \in \mathbb{R}^{k \times n}$$

These k directions capture the most variance. The choice of k is determined by the explained variance ratio (see below).

PCA: Projection, Reconstruction, and Reconstruction Error

Step 4 — Project (Encode into Low Dimensions)

$$\mathbf{Z} = \mathbf{X}_c \mathbf{P}_k^\top \in \mathbb{R}^{m \times k}$$

Row i of $\mathbf{Z}$: $\mathbf{z}_i = \mathbf{P}_k \mathbf{x}_{c,i} \in \mathbb{R}^k$ gives the PCA scores (coordinates in the new basis) for image i .

The j -th score $z_{ij} = \mathbf{p}_j^\top \mathbf{x}_{c,i}$ is the dot product of eigenface j with the centred image — it measures “how much of eigenface j is present in image i ”.

Step 5 — Reconstruct (Decode Back to Original Space)

Given scores $\mathbf{z}_i$, reconstruct the (approximate) original image:

$$\hat{\mathbf{x}}_i = \boldsymbol{\mu} + \mathbf{z}_i \mathbf{P}_k = \boldsymbol{\mu} + \mathbf{x}_{c,i} \mathbf{P}_k^\top \mathbf{P}_k$$

In the most interpretable summation form:

$$\hat{\mathbf{x}}_i = \boldsymbol{\mu} + \sum_{j=1}^k \underbrace{(\mathbf{p}_j^\top \mathbf{x}_{c,i})}_{z_{ij}=\text{score}} \mathbf{p}_j$$

Reading this: Start at the mean face. For each eigenface $\mathbf{p}_j$, compute how much of it is present in image i (score z_{ij}), then add back that eigenface weighted by its score.

Note: $\mathbf{P}_k^\top \mathbf{P}_k \neq \mathbf{I}_n$

$\mathbf{P}_k \in \mathbb{R}^{k \times n}$ has orthonormal rows, so $\mathbf{P}_k \mathbf{P}_k^\top = \mathbf{I}_k$ (the $k \times k$ identity). But $\mathbf{P}_k^\top \mathbf{P}_k \in \mathbb{R}^{n \times n}$ is only a rank- k projection — it projects onto the k -dimensional subspace. Reconstruction is therefore an approximation when $k < n$.

Explained Variance Ratio and Choosing k

How Much Information Does Each Component Capture?

We showed that the variance of the data along $\mathbf{p}_i$ equals the eigenvalue λ_i . The **total variance** in the data is $\sum_{j=1}^n \lambda_j$.

The **Explained Variance Ratio** for component i :

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^n \lambda_j}$$

This gives the fraction of total variance captured by direction $\mathbf{p}_i$. Since the λ_j are ordered (descending), the first component always captures the most, the second the second most, and so on.

Choosing k : find the smallest k such that $\sum_{i=1}^k \text{EVR}_i \geq \text{threshold}$ (commonly 0.95 or 0.99).

Application-Dependent Threshold (From Lecture)

Set aside 100 validation images. Compute reconstruction error for $k = 100, 1000, 10,000$. The right k depends on your application:

- **Face recognition:** high reconstruction error may still be acceptable
- **Industrial defect detection:** a single anomalous pixel must be preserved — much lower reconstruction error required

Have a domain expert define what “acceptable” means.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)**
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

Why Rectangular Matrices Are Harder: The Eigenvector Problem

Recap: What Made Eigenvectors Useful

For a square matrix $A \in \mathbb{R}^{n \times n}$ with eigenvectors forming a basis, the expensive matrix multiplication $A\mathbf{x}$ reduced to cheap scalar multiplications: $A\mathbf{x} = \sum_i \alpha_i \lambda_i \mathbf{v}_i$.

Can we replicate this advantage for rectangular matrices?

The Fundamental Obstruction

The eigenvalue equation $A\mathbf{v} = \lambda\mathbf{v}$ requires both sides to live in the *same* vector space. For a **rectangular** $A \in \mathbb{R}^{m \times n}$ with $m \neq n$:

$$\underbrace{A}_{\in \mathbb{R}^n} \underbrace{\mathbf{v}}_{\in \mathbb{R}^n} = \underbrace{\mathbf{w}}_{\in \mathbb{R}^m}$$

The output lives in $\mathbb{R}^m$, the input in $\mathbb{R}^n$. The equation $A\mathbf{v} = \lambda\mathbf{v}$ cannot hold because $\lambda\mathbf{v} \in \mathbb{R}^n$ while $A\mathbf{v} \in \mathbb{R}^m$.

Rectangular matrices cannot have eigenvectors. They are the “super-villains” of linear algebra: they take a vector from one space and dump it into a completely different space, destroying its identity.

Why This Matters in Practice

Data matrices $X \in \mathbb{R}^{m \times n}$ are almost always rectangular. SVD is the resolution: it finds two sets of orthonormal vectors — one for the input space $\mathbb{R}^n$ and one for the output space $\mathbb{R}^m$ — that together restore the scalar-multiplication advantage.

SVD: The Core Idea — Paired Bases in Two Spaces

A Motivated Setup

Since one basis cannot work for both sides, suppose we find *two* orthonormal bases:

- $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$ (orthonormal, basis for the *row space* / inputs of A)
- $\mathbf{u}_1, \dots, \mathbf{u}_k \in \mathbb{R}^m$ (orthonormal, basis for the *column space* / outputs of A)

And suppose they satisfy the “scalar-like” relation:

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i, \quad i = 1, \dots, k$$

where $\sigma_i > 0$ are real scalars called **singular values**.

Why This Restores the Scalar-Multiplication Advantage

Write any input as $\mathbf{x} = \sum_{i=1}^k \alpha_i \mathbf{v}_i$. Then:

$$A\mathbf{x} = \sum_{i=1}^k \alpha_i A\mathbf{v}_i = \sum_{i=1}^k \alpha_i \sigma_i \mathbf{u}_i$$

Once again, matrix multiplication reduced to scalar multiplication ($\alpha_i \mapsto \alpha_i \sigma_i$) — this time valid for a *rectangular* matrix that maps between two different spaces. The σ_i play the role of eigenvalues, but operate between two separate bases.

SVD: Matrix Form and Full Statement

Writing $A\mathbf{v}_i = \sigma_i\mathbf{u}_i$ as a Matrix Equation

Stack relations simultaneously (for all $i = 1, \dots, k$):

$$A[\mathbf{v}_1 \cdots \mathbf{v}_k] = [\sigma_1\mathbf{u}_1 \cdots \sigma_k\mathbf{u}_k] = [\mathbf{u}_1 \cdots \mathbf{u}_k] \text{diag}(\sigma_1, \dots, \sigma_k)$$

Extend $\mathbf{v}$'s to a full basis of $\mathbb{R}^n$ (null-space directions added via Gram–Schmidt) and $\mathbf{u}$'s to a full basis of $\mathbb{R}^m$. Let $V = [\mathbf{v}_1 \cdots \mathbf{v}_n] \in \mathbb{R}^{n \times n}$, $U = [\mathbf{u}_1 \cdots \mathbf{u}_m] \in \mathbb{R}^{m \times m}$, $\Sigma \in \mathbb{R}^{m \times n}$ diagonal with $\Sigma_{ii} = \sigma_i$. Then:

$$AV = U\Sigma \quad \Rightarrow \quad \boxed{A = U\Sigma V^T}$$

This is the **Singular Value Decomposition** of A . Since U and V are orthogonal ($UU^T = I$, $VV^T = I$), we have $V^{-1} = V^T$, $U^{-1} = U^T$.

Dimensions of Each Factor

$U \in \mathbb{R}^{m \times m}$	—	left singular vectors (column space)
$\Sigma \in \mathbb{R}^{m \times n}$	—	singular values on diagonal
$V \in \mathbb{R}^{n \times n}$	—	right singular vectors (row space)

Economy (thin) SVD: Keep only the $k = \text{rank}(A)$ non-zero singular values — $U_k \in \mathbb{R}^{m \times k}$, $\Sigma_k \in \mathbb{R}^{k \times k}$, $V_k \in \mathbb{R}^{n \times k}$.

How to Find U and V : Reduction to EVD

Finding V (Right Singular Vectors)

Assume $A = U\Sigma V^T$ exists. Compute $A^T A$:

$$\begin{aligned} A^T A &= (U\Sigma V^T)^T (U\Sigma V^T) \\ &= V\Sigma^T U^T U\Sigma V^T \\ &= V \underbrace{\Sigma^T \Sigma}_{\text{diagonal, entries } \sigma_i^2} V^T \end{aligned}$$

(using $U^T U = I$ since U is orthogonal).

This is exactly the EVD of $A^T A$: V **consists of the eigenvectors of $A^T A$** and the eigenvalues are σ_i^2 .

Finding U (Left Singular Vectors) and the Singular Values

Similarly, compute AA^T :

$$AA^T = (U\Sigma V^T)(V\Sigma^T U^T) = U \underbrace{\Sigma \Sigma^T}_{\text{diagonal, entries } \sigma_i^2} U^T$$

So U **consists of the eigenvectors of AA^T** .

Both $A^T A$ and AA^T share the same non-zero eigenvalues (the same rank identity we used for the Gram matrix!), and the **singular values are their square roots**:

$$\sigma_i = \sqrt{\lambda_i(A^T A)} = \sqrt{\lambda_i(AA^T)}$$

Why SVD Always Exists (But EVD Does Not)

Existence Conditions: EVD vs SVD

EVD of A requires:

- 1 A is **square** ($n \times n$)
- 2 A has n linearly independent eigenvectors (not always true — e.g. defective matrices)

SVD has no such restrictions. It always exists because:

- $A^T A$ is always **real, symmetric, and PSD** regardless of the shape of A
- By the Spectral Theorem, every real symmetric PSD matrix has a complete set of real orthogonal eigenvectors
- Therefore V (eigenvectors of $A^T A$) always exists
- Similarly U (eigenvectors of AA^T) always exists

Every real matrix $A \in \mathbb{R}^{m \times n}$ (any shape, any rank) has an SVD.

Geometric Interpretation

Any linear map $\mathbb{R}^n \rightarrow \mathbb{R}^m$ decomposes into exactly three steps: **(1)** rotate/reflect in the input space (by V^T); **(2)** scale along coordinate axes (by Σ); **(3)** rotate/reflect in the output space (by U). SVD reveals this “rotate–scale–rotate” structure for any matrix.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem**
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

SVD as a Sum of Rank-1 Matrices

The Outer Product Expansion

The SVD $A = U\Sigma V^T$ can be written as a sum of k rank-1 matrices (outer products):

$$A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \cdots + \sigma_k \mathbf{u}_k \mathbf{v}_k^T = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

Why is each term $\sigma_i \mathbf{u}_i \mathbf{v}_i^T$ rank-1? An outer product $\mathbf{u}\mathbf{v}^T$ (a column times a row) has rank exactly 1 — every column is a multiple of $\mathbf{u}$. Adding k rank-1 matrices gives at most rank k .

The terms are ordered by *importance*: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k \geq 0$. The first term has the largest weight; the last term has the smallest.

Dropping the last $k - r$ terms gives a rank- r approximation $A_r = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$.

Storage: Original vs. Compressed

Original A : $m \times n$ entries. Rank- r approximation: store r triples $(\mathbf{u}_i, \sigma_i, \mathbf{v}_i)$: $(m + 1 + n) \times r$ values.

Example: $m = 400$, $n = 10,000$, $r = 50$: original = 4×10^6 values; compressed $\approx 5 \times 10^5$ values — an $8\times$ reduction with minimal quality loss.

The Eckart–Young–Mirsky Theorem

Theorem 1 (Best Low-Rank Approximation)

Let $A \in \mathbb{R}^{m \times n}$ with SVD $A = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$. The **best rank- r approximation** in the Frobenius norm is:

$$A_r = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

meaning A_r solves:

$$\min_{B: \text{rank}(B) \leq r} \|A - B\|_F^2$$

The minimum error achieved is: $\|A - A_r\|_F^2 = \sum_{i=r+1}^k \sigma_i^2$.

Deep Meaning of This Theorem

If you must represent A with limited rank and want to minimise total squared entry-wise error, the answer is exactly the first r terms of the SVD — nothing else can do better. The singular values σ_i directly quantify reconstruction quality: dropping term i costs exactly σ_i^2 in Frobenius-norm error. So you should always drop the **smallest** singular values first.

Connection to the Lecture

The professor's remark: "if you are trying to reconstruct the original matrix and pick the top k singular values, that is the *best possible* reconstruction" — this is precisely the Eckart–Young theorem stated informally.

The Frobenius Norm: What We Are Minimising

Definition and Intuition

The Frobenius norm of $A \in \mathbb{R}^{m \times n}$:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(A^\top A)}$$

It is the generalisation of the Euclidean vector norm to matrices: just flatten the matrix into a long vector and take the Euclidean norm.

$\|A - B\|_F^2 = \sum_{i,j} (a_{ij} - b_{ij})^2$ measures total squared entry-wise difference — the natural way to quantify how close two matrices are.

Connection to Singular Values

Using $\text{tr}(A^\top A) = \text{tr}(V \Sigma^2 V^\top) = \text{tr}(\Sigma^2)$ (trace is invariant under cyclic permutations):

$$\|A\|_F^2 = \sum_{i=1}^k \sigma_i^2$$

The singular values completely determine the Frobenius norm. Dropping terms $r+1, \dots, k$ from the SVD expansion loses exactly $\sum_{i=r+1}^k \sigma_i^2$ of “energy” — the smallest contributions.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD**
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

Row Space, Column Space, Null Space, and Rank

The Four Fundamental Subspaces of $A \in \mathbb{R}^{m \times n}$

- **Column space** (range): $\text{col}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$. Dimension = $\text{rank}(A) = k$.
- **Row space**: $\text{col}(A^\top) \subseteq \mathbb{R}^n$. Dimension = $\text{rank}(A) = k$.
- **Null space**: $\text{null}(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\} \subseteq \mathbb{R}^n$. Dimension = $n - k$ (Rank–Nullity Theorem: $\text{rank}(A) + \text{nullity}(A) = n$).
- **Left null space**: $\text{null}(A^\top) \subseteq \mathbb{R}^m$. Dimension = $m - k$.

How SVD Reveals All Four Subspaces at Once

- $\mathbf{v}_1, \dots, \mathbf{v}_k$ (right sing. vectors with $\sigma_i > 0$) span the **row space** of A
- $\mathbf{v}_{k+1}, \dots, \mathbf{v}_n$ span the **null space** of A (since $A\mathbf{v}_i = \sigma_i \mathbf{u}_i = \mathbf{0}$ when $\sigma_i = 0$)
- $\mathbf{u}_1, \dots, \mathbf{u}_k$ span the **column space**
- $\mathbf{u}_{k+1}, \dots, \mathbf{u}_m$ span the **left null space**

Key consequence: Only the k -dimensional subspace (the row space) of $\mathbb{R}^n$ can produce a non-zero output when multiplied by A . Null-space vectors are completely destroyed. Hence we need only k basis vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ to fully describe what A does — the remaining $n - k$ dimensions are invisible to A .

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection**
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections

PCA via Direct SVD of the Data Matrix

A More Direct Route to PCA

Instead of computing $\Sigma = \frac{1}{m} X_c^\top X_c$ and then doing its EVD, we can apply SVD directly to the centred data matrix $X_c \in \mathbb{R}^{m \times n}$:

$$X_c = U_X \Sigma_X V_X^\top$$

Now compute $X_c^\top X_c$:

$$X_c^\top X_c = (U_X \Sigma_X V_X^\top)^\top (U_X \Sigma_X V_X^\top) = V_X \Sigma_X^\top U_X^\top U_X \Sigma_X V_X^\top = V_X \Sigma_X^2 V_X^\top$$

Comparing with the EVD $\Sigma_{\text{cov}} = P \Lambda P^\top$:

- **Principal directions:** $P = V_X$ (right singular vectors of X_c)
- **Eigenvalues:** $\lambda_i = \sigma_{X,i}^2 / m$

PCA is equivalent to SVD of the centred data matrix.

Why This Is Numerically Better

Computing $X_c^\top X_c$ squares the condition number of X_c , amplifying numerical errors. Running SVD directly on X_c avoids this and is generally more stable. In practice, `sklearn.decomposition.PCA` uses SVD internally, not the covariance matrix EVD.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition**
- 10 Summary: SVD, PCA, and Their Connections

Eigenfaces: The Full Pipeline

Data Setup

m training face images, each 100×100 pixels $\Rightarrow n = 10,000$ features per image. Data matrix $X \in \mathbb{R}^{m \times 10,000}$.

After centring: X_c with zero-mean columns. Direct EVD of $\Sigma \in \mathbb{R}^{10,000 \times 10,000}$ is infeasible.

Computing Eigenfaces via the Gram Matrix Trick

- 1 Compute $G = \frac{1}{m} X_c X_c^T \in \mathbb{R}^{m \times m}$ (e.g. 400×400 — feasible)
- 2 Solve $G \mathbf{v}_i = \lambda_i \mathbf{v}_i$: only $O(m^3)$ cost
- 3 Recover eigenfaces: $\mathbf{p}_i = \frac{X_c^T \mathbf{v}_i}{\sqrt{m \lambda_i}} \in \mathbb{R}^{10,000}$
- 4 Reshape each $\mathbf{p}_i$ to 100×100 for visualisation — these “ghost-like” images are the **eigenfaces**
- 5 Keep top- k eigenfaces ($k \approx 50$ – 100 is often enough)

Key Properties

- Eigenfaces form an orthonormal basis for the “face subspace”
- Any face $\approx \boldsymbol{\mu} + \sum_{i=1}^k z_i \mathbf{p}_i$
- Store $\mathbf{p}_1, \dots, \mathbf{p}_k$ once; only k scalars z_i needed per new image
- Multiple images of the same person (different lighting) map to nearby points in the k -dimensional code space

Eigenfaces: Compression Analysis and Generalisation

Storage Analysis

Original dataset: $m \times n = m \times 10,000$ numbers. After PCA with k components, we store:

$$\underbrace{k \times n}_{\text{eigenfaces } P_k} + \underbrace{n}_{\text{mean face } \mu} + \underbrace{m \times k}_{\text{scores } Z} \text{ numbers}$$

Concrete example ($m = 400$, $n = 10,000$, $k = 50$):

- Original: $400 \times 10,000 = 4,000,000$ values
- Compressed: $50 \times 10,000 + 10,000 + 400 \times 50 = 510,000$ values
- Compression factor: $\approx 7.8\times$ — almost $8\times$ storage reduction

Generalisation: Does It Work on New Faces?

Any new face can be projected onto the learned eigenface basis and approximately reconstructed — as long as the training data was sufficiently diverse.

If training data is narrow (e.g. only one demographic), faces from other demographics will be reconstructed poorly, because the basis does not span the relevant directions.

The same principle as any ML model: train on representative data, and it generalises; train on narrow data, and it fails on out-of-distribution inputs.

Current Section

- 1 Variance, Covariance, and the Covariance Matrix
- 2 Eigenvalue Decomposition (EVD) and the Spectral Theorem
- 3 The Gram Matrix Trick: Avoiding $O(n^3)$ Cost
- 4 The Full PCA Pipeline
- 5 Singular Value Decomposition (SVD)
- 6 Low-Rank Approximation and the Eckart–Young Theorem
- 7 Fundamental Subspaces via SVD
- 8 SVD and PCA: The Complete Connection
- 9 Eigenfaces: PCA Applied to Face Recognition
- 10 Summary: SVD, PCA, and Their Connections**

Summary: The Big Picture

EVD (Square Matrices Only)

$$A = V\Lambda V^{-1}$$

$$\text{Symmetric: } A = V\Lambda V^T$$

Requires: square; n independent eigenvectors

Gives: eigenvalues λ_i , eigenvectors $\mathbf{v}_i$

$$A\mathbf{v}_i = \lambda_i\mathbf{v}_i$$

SVD (Any Matrix)

$$A = U\Sigma V^T$$

U : left singular vectors (= eigenvectors of AA^T)

V : right singular vectors (= eigenvectors of $A^T A$)

$\sigma_i = \sqrt{\lambda_i}$: singular values

Always exists. No shape restriction.

PCA (via EVD or SVD)

Goal: Low-dimensional representation

Tool: EVD of $\Sigma = \frac{1}{m}X_c^T X_c$

Shortcut 1: SVD of X_c directly

Shortcut 2: EVD of $G = \frac{1}{m}X_c X_c^T$ (Gram trick)

Principal directions = eigenvectors of Σ

Variance along $\mathbf{p}_i$ = eigenvalue λ_i

Three Views of PCA

1. Decorrelate: zero off-diagonal covariance
2. Max variance: keep highest- λ directions
3. Min error: discard lowest- λ directions

*“SVD always exists. EVD needs square matrices.
PCA is EVD of the covariance matrix — or equivalently,
SVD of the data matrix.”*

Thank You

Questions?